

Esercizio A1 – Complessità – Punti ≥ 8

Le tre funzioni di ricorrenza possono essere scritte in questo modo:

$$\begin{aligned}T_A(n) &= 5T_A(n/2) + n \\T_B(n) &= 2T_B(n-2) + 1 \\T_C(n) &= 9T(n/3) + n^2\end{aligned}$$

Utilizzando le varie versioni del teorema dell'esperto, otteniamo:

$$\begin{aligned}T_A(n) &= \Theta(n^{\log_2 5}) \\T_B(n) &= \Theta(2^n) \\T_C(n) &= \Theta(n^2 \log n)\end{aligned}$$

Ovviamente, l'algoritmo B è da escludere; l'esponente di $T_A(n)$ è maggiore di 2, quindi l'algoritmo C è quello che ha complessità inferiore.

Esercizio A2 – Alberi isobati – Punti ≥ 10

Si noti che se un albero è isobato, allora i sottoalberi radicati nei figli della radice sono anch'essi isobati. Per assurdo, supponiamo che non sia così; allora in uno di essi ci sarebbero due foglie con profondità p_1 e p_2 nel sottoalbero, con $p_1 \neq p_2$. Queste due foglie avrebbero profondità $p_1 + 1 \neq p_2 + 1$ nell'albero considerato, rendendolo non isobato, il che è assurdo.

Si potrebbe calcolare la profondità delle foglie utilizzando un parametro aggiuntivo che viene incrementato a ogni chiamata; per semplificare, dato un nodo t , scriviamo una funzione ausiliaria che, applicata a un nodo t , restituisce:

- la distanza dalle sue foglie, se è isobato;
- -1 se il sottoalbero radicato è vuoto, ovvero $t = \mathbf{nil}$;
- $-\infty$ se il sottoalbero radicato non è isobato e quindi le sue foglie hanno distanze diverse.

A parte il caso in cui $t = \mathbf{nil}$, la funzione ausiliaria viene richiamata sul figlio sinistro e sul figlio destro, memorizzando i risultati in p_L e p_R ; dopodiché, si gestiscono i vari casi possibili. Si noti che il caso delle foglie è gestito dalla condizione $p_R = p_L$, che somma 1 al valore $p_L = -1$ e ottiene la corretta distanza 0.

```
boolean isIsobath(TREE T)
```

```
    return helper(T) == ∞
```

```
int helper(TREE t)
```

```
    if t == nil then
```

```
        | return -1
```

```
    else
```

```
        int pL = helper(t.left)
```

```
        int pR = helper(t.right)
```

```
        if pL == -∞ or pR == -∞ then
```

```
            | return -∞
```

% Uno dei due sottoalberi non è isobato

```
        else if pL == -1 then
```

```
            | return pR + 1
```

% Solo figlio destro

```
        else if pR == -1 then
```

```
            | return pL + 1
```

% Solo figlio sinistro

```
        else if pR == pL then
```

```
            | return pL + 1
```

% Isobato, aumento la distanza delle foglie di 1

```
        else
```

```
            | return -∞
```

Essendo una visita in profondità, il costo della funzione è $O(n)$.

Esercizio A3 – Where’s negative? – Punti ≥ 12

Una soluzione banale consiste in un semplice ciclo **while** che scorre tutti i valori naturali da 1 in poi, fino a incontrare il primo valore n tale per cui $f(n) < 0$. Tale algoritmo ha costo lineare e viene valutato 30%.

Per migliorare questa soluzione, possiamo utilizzare un’idea simile al metodo della bisezione per trovare gli zeri di un funzione, che prevede di considerare un intervallo $start, end$, dove $f(start) > 0$ e $f(end) < 0$, e continuare a restringere l’intervallo fino a quando si identifica il primo valore negativo. A differenza del caso continuo, il nostro algoritmo termina sempre con un valore preciso.

Non conoscendo a priori un valore end tale che $f(end)$ sia minore di zero, dobbiamo cercarlo. Utilizzeremo una ricerca esponenziale, che consiste nell’inizializzare il valore end a 1 e raddoppiarlo progressivamente finché $f(end)$ non diventa negativo. Questo processo richiede un numero di iterazioni proporzionale a $\log n$, dato che ogni passo raddoppia l’indice end fino a superare il punto in cui f cambia segno.

Una volta identificati un indice end per cui $f(end) < 0$ e un indice $start$ per cui $f(start) \geq 0$ (andrebbe bene $start = 1$, ma sappiamo che $start = end/2$ è sicuramente positivo), procediamo con una ricerca dicotomica. L’obiettivo è ridurre l’intervallo tra $start$ e end finché end non è l’elemento immediatamente successivo a $start$, e dunque end sarà l’indice cercato, il più piccolo per cui $f(n) < 0$. Anche la ricerca dicotomica avrà costo computazionale $O(\log n)$.

```
int findNegative()
    int end = 1
    while f(end) ≥ 0 do
        end = end · 2
    return fnRec(end/2, end)
```

```
int fnRec(int start, int end)
    if end = start + 1 then
        return end
    int mid = (start + end)/2
    if f(mid) < 0 then
        return fnRec(start, mid)
    else
        return fnRec(mid, end)
```

Esercizio B1 – Connect-x – Punti ≥ 8

L’algoritmo può essere facilmente risolto in tempo $O(n^3)$, cercando la sequenza più lunga che parte da ogni casella nelle due direzioni.

Per risolvere il problema in tempo $\Theta(n^2)$, è possibile semplicemente scorrere ogni riga e ogni colonna, considerando la più lunga sequenza consecutiva che si incontra e aggiornando un variabile *maxSoFar* che verrà poi infine restituita. Potremmo chiamarla programmazione dinamica, ma direi che più semplicemente è buon senso.